

## Session 4 · Deck 4.B4

# Fast and Accurate: The Same Model, Measured

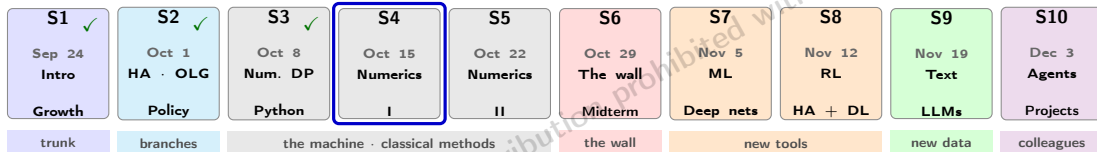
Zhigang Feng

Thursday, October 15, 2026

*Build it on paper → solve it on the machine → push it to the frontier with AI*

# Where We Are

you are here: 4.B4 ▼



*Build it on paper → solve it on the machine → push it to the frontier with AI*

**Block B:** 4.B1 approximation and interpolation · 4.B2 quadrature · 4.B3 discretizing an AR(1) · **4.B4** fast and accurate · 4.B5 the worked example, end to end.

# One Model, Six Solvers

Quarterly RBC,  $n_k = 500$ ,  $n_z = 7$ , tolerance  $10^{-6}$ , identical in every row.

|                       | time          | speed-up    | worst $\log_{10} \varepsilon$ |
|-----------------------|---------------|-------------|-------------------------------|
| plain VFI             | 3.46 s        | 1.0         | -1.55                         |
| + Howard ( $n_H=20$ ) | 0.25 s        | 14.1        | -1.55                         |
| + Howard ( $n_H=50$ ) | 0.16 s        | 22.1        | -1.55                         |
| + monotonicity        | 1.10 s        | 3.2         | -1.55                         |
| + continuous choice   | 1.11 s        | 3.1         | -2.77                         |
| <b>EGM</b>            | <b>0.09 s</b> | <b>37.7</b> | <b>-6.06</b>                  |

## Read the two columns separately

The acceleration rows are **faster at identical accuracy**. The last two rows are **more accurate**. Only one row is both.

EGM is 38 times faster *and* four and a half decades more accurate — and it does zero root-finding to get there.

# Where the Time Actually Goes

The plain solver's bill factorises:

$$\underbrace{n_k n_z n_k}_{\text{one sweep}} \times \underbrace{\frac{\ln \varepsilon}{\ln \beta}}_{\text{patience}} = 1.75\text{M} \times 1352 = \mathbf{2.4 \text{ billion.}}$$

Two independent things to attack:

- **The sweep count**, which belongs to  $\beta$  alone — 1,352 measured against a theoretical 1,375, at  $\beta = 0.99$ .
- **The cost per sweep**, which is the maximization over  $n_k$  candidates at every one of  $n_k n_z$  states.

## This got worse since Session 3

The same solver needed **228** sweeps in 3.A4 at  $\beta = 0.95$ . Moving to a quarterly calibration multiplied the patience bill by six, and nothing about the economics changed.

Howard attacks the first. Monotonicity and concavity attack the second. **EGM removes the maximization entirely.**

Cost accounting adapted from QM Lec. 8, "Recap: Standard VFI" and "Computational Bottlenecks".

# Howard: Stop Maximizing So Often

The policy settles long before the value does — 3.A1 measured it, at iteration 11 out of 272. So most sweeps re-derive a policy that has not changed.

**Howard's policy improvement** alternates:

- one *improvement* step: the full max,  $O(n_k^2 n_z)$ ;
- then  $n_H$  *evaluation* steps, which reuse the stored policy index and cost only  $O(n_k n_z)$ :

$$v^{(n+1)}(k_i, z_j) = u(c_{ij}) + \beta \sum_m \Pi_{jm} v^{(n)}(g(k_i, z_j), z_m).$$

No argmax — just a lookup.

## Why it converges

Policy evaluation solves a *linear* fixed point,  $v_g = (I - \beta P_g)^{-1} u_g$ . Iterating it  $n_H$  times is itself a contraction with modulus  $\beta$ , so you land within  $\beta^{n_H}$  of  $v_g$ .

The improvement step is monotone and there are finitely many grid policies, so the algorithm terminates in finitely many policy updates — something plain VFI cannot promise.

Adapted from QM-Lec. 8, "Howard's Policy Improvement" block. Step count written  $n_H$ ; the source uses  $H$ , which this course reserves for the law of motion.

# Howard, Measured

|            | policy updates | comparisons | time          |
|------------|----------------|-------------|---------------|
| plain VFI  | 1,352          | 2.37 bn     | 3.46 s        |
| $n_H = 10$ | 124            | 221 m       | 0.39 s        |
| $n_H = 20$ | 66             | 120 m       | 0.25 s        |
| $n_H = 50$ | <b>29</b>      | <b>56 m</b> | <b>0.16 s</b> |

From 1,352 maximizations to **29**, at *identical* accuracy — every row has the same worst Euler error,  $10^{-1.55}$ .

Rule of thumb from the source:  $n_H \in [10, 50]$ . Our measurement supports it.

## Diminishing returns

$n_H : 10 \rightarrow 20$  halves the updates.  $20 \rightarrow 50$  halves them again but only cuts the time by a third, because the evaluation steps are no longer free.

**Accuracy did not move.** Howard changes how you reach the fixed point, not which fixed point it is. The discretization error is untouched.

# Monotonicity: Fewer Comparisons, More Wall Clock

$g(k, z)$  is increasing in  $k$ , so the maximizer at  $k_{i+1}$  cannot lie below the maximizer at  $k_i$ . Search upward from wherever the previous state stopped, and the sweep falls from  $O(n_k^2)$  to  $O(n_k)$ .

| Howard $n_H = 20$    | comparisons | time   |
|----------------------|-------------|--------|
| without monotonicity | 120 m       | 0.25 s |
| with monotonicity    | 64 m        | 1.10 s |

Half the comparisons, and four and a half times slower.

## Both numbers are true

The operation count is what the theory predicts. The wall clock is what the machine does — and a monotone ratchet is a *sequential* loop, while the full search is one vectorised array operation.

**An algorithmic improvement is not automatically a speed-up.** Report the operation count, which is implementation-independent, *and* the wall clock, which is what you actually wait for. In a compiled language this row goes the other way.

# Concavity, and the Callback to This Morning

Concavity of  $u$  and of the interpolated  $\hat{v}$  makes the inner objective **unimodal**, which is exactly the licence golden section needs — and 4.B1 established that linear interpolation is what supplies it.

So the inner maximization can be replaced by 4.A2's bracket-shrinking, at  $O(\log(1/\varepsilon))$  evaluations instead of  $O(n_k)$  comparisons, *and* the answer is no longer confined to a grid point.

Measured: the Euler error falls from  $10^{-1.55}$  to  $10^{-2.77}$ , and the objective-evaluation count falls from 2.4 billion to **18.9 million**.

Adapted from QM Lec. 8, "Exploiting Concavity" and "Golden Section Search in VF1". The source's "Recall Lecture 4" does not apply here — golden section was deliberately withheld until 4.A2.

## Two gains, one change

Monotonicity and Howard made the *same* answer cheaper. Continuous choice makes a *different*, better answer — it moves the accuracy, not just the clock.

Combine them: use monotonicity to bracket, then golden section inside the bracket, then Howard around the whole thing.

# The Endogenous Grid Method: Run the Euler Equation Backwards

Time iteration is expensive because  $k'$  appears on *both* sides of

$$u'(y(k, z) - k') = \beta \mathbb{E}[u'(c(k', z'))R(k', z')],$$

so every state needs a root-find, every sweep.

**Carroll's insight (2006):** treat the grid as *tomorrow's* assets. Fix  $k'$ . Then the right-hand side is computable immediately, and the left-hand side inverts:

$$c = (u')^{-1}\left(\beta \mathbb{E}[u'(c(k', z'))R(k', z')]\right).$$

Now you know  $c$  and  $k'$  — and today's state follows.

## The trade

The unknown moves from *inside* a non-linear equation to the *outside* of an explicit formula. For CRRA,  $(u')^{-1}(x) = x^{-1/\sigma}$  — one exponentiation.

The grid you end up with is **endogenous**: the  $k$  values that generate your chosen  $k'$  are wherever the algebra puts them. One interpolation brings the policy back to the grid you wanted.

Carroll, "The method of endogenous gridpoints for solving dynamic stochastic optimization problems", *Economics Letters* (2006).

# Getting EGM Right: Put the Grid on Cash on Hand

Almost every presentation of EGM says “no root-finding required” — and then, with Cobb–Douglas production, has to solve

$$f(\tilde{k}) + (1 - \delta)\tilde{k} - c - k' = 0$$

for today's capital  $\tilde{k}$ . That is a root-find, at every node, every sweep. The headline claim is retracted three slides after it is made.

**The fix is to choose the right state.** Let

$$m = zk^\alpha + (1 - \delta)k \quad (\text{cash on hand}).$$

Then the budget constraint is simply  $m = c + k'$ , so

$$\tilde{m} = c + k',$$

with no equation to solve at all.

## Why this is the real method

Carroll's formulation is in cash on hand for exactly this reason. Keeping the grid in  $k$  reintroduces the root-find the method exists to remove.

Measured consequence: our cash-on-hand EGM does **zero** root solves and runs in 0.09 s. An implementation that keeps the grid in  $k$  does 3,500 root solves per sweep and loses the speed race it was supposed to win.

# EGM in Four Steps

For each  $z_j$  and each end-of-period  $k'$  on the exogenous grid:

1. **Next period's cash** for every  $z'$ :  $m' = z'(k')^\alpha + (1 - \delta)k'$ , and  $c' = \hat{c}(m', z')$  by interpolation.
2. **The expectation**:  $\mathcal{R} = \beta \sum_m \Pi_{jm} u'(c') R(k', z_m)$ .
3. **Invert**:  $c = (u')^{-1}(\mathcal{R})$ . For log utility,  $c = 1/\mathcal{R}$ .
4. **Today's state**:  $\tilde{m} = c + k'$ . Interpolate  $(\tilde{m}, c)$  back onto the  $m$ -grid.

Where  $m$  falls below the smallest  $\tilde{m}$ , the constraint binds and  $c = m - \underline{k}$ .

Steps adapted from QM Lec. 8's EGM block, restated on cash on hand.

## Count the solves

Steps 1–4 contain one interpolation, one matrix product, one exponentiation and one addition. **No maximization and no root-finding.**

That is the whole method. It is shorter than the VFI loop it replaces, and it converged here in 420 iterations and 0.09 seconds.

# When EGM Does Not Apply

The method needs four things, and each is a real restriction:

1.  $u'$  must be **invertible** in closed form — CRRA, CARA, quadratic. Not satiation.
2. The budget must solve for today's state **explicitly** given  $(c, k')$  — which is what cash on hand buys.
3. **One** continuous control. Two, and part of the problem goes back to root-finding.
4. The endogenous grid must come out **ordered**, which needs a monotone policy.

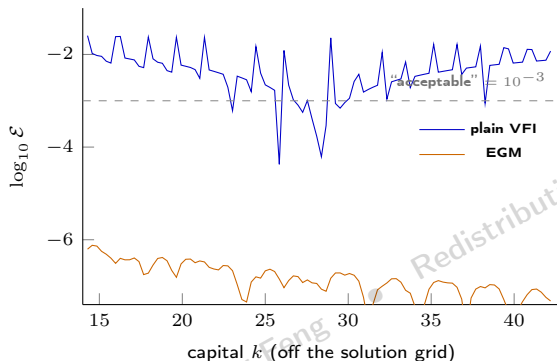
## What breaks item 4

Discrete choices — to work or not, to default or not — make the policy non-monotone, the endogenous grid folds over itself, and a plain interpolation picks the wrong branch.

The repair is an **upper-envelope** step: Fella (2014), and Iskhakov, Jørgensen, Rust & Schjerning (2017) for the discrete-continuous case. Both are outside this course, and both are what you would read next.

Conditions adapted from QM Lec. 8, "When It Works and When It Doesn't"; the DC-EGM reference is added.

# The Accuracy, Where the Source Has No Figure



Unit-free Euler residuals at 400 capital levels *between* the solution grid points, at the middle productivity state.

|           | mean         | worst        |
|-----------|--------------|--------------|
| plain VFI | -2.18        | -1.55        |
| EGM       | <b>-6.75</b> | <b>-6.06</b> |

**Four and a half decades**, on the same grid, the same chain and the same tolerance. VFI's worst case is a 2.8% consumption mistake; EGM's is one part in a million.

QM Lec. 8's "Euler Errors: Visualization" frame describes this plot in words and contains no figure. This one is drawn from our run.

# Reporting: What Goes in the Paper

1. The method, and the implementation choices that matter.
2. The grid: points, spacing, bounds — and the chain, with its *implied*  $\rho$  and  $\sigma_z$  (4.B3).
3. The convergence tolerance.
4. **Maximum and mean Euler errors on an evaluation grid that is not the solution grid.**
5. A grid-density study, and whether the observed rate matches theory.
6. Computation time and hardware. ●

## Item 4 is the one with teeth

Evaluating residuals *on* the solution grid is the numerical analogue of reporting training accuracy. The solution was constructed to satisfy the Euler equation there.

And item 5 has to name which object the rate applies to. 3.A2 measured **1.89 for the value function** and **0.93 for the policy** — quoting a single rate without saying which is how the two get confused.

Checklist adapted from QM Lec. 8, “Reporting Standards”; Den Haan–Marcet is the simulation-based complement, and needs a stochastic model to be non-degenerate.

# The Benchmark Table, Recomputed

| Method                   | time          | speed-up    | worst $\log_{10} \mathcal{E}$ | what changed                     |
|--------------------------|---------------|-------------|-------------------------------|----------------------------------|
| Plain VFI, grid search   | 3.46 s        | 1.0         | -1.55                         | —                                |
| + Howard, $n_H = 20$     | 0.25 s        | 14.1        | -1.55                         | fewer maximizations              |
| + Howard, $n_H = 50$     | 0.16 s        | 22.1        | -1.55                         | fewer still                      |
| + monotonicity           | 1.10 s        | 3.2         | -1.55                         | fewer comparisons, more overhead |
| + continuous choice      | 1.11 s        | 3.1         | -2.77                         | the policy leaves the grid       |
| <b>EGM, cash on hand</b> | <b>0.09 s</b> | <b>37.7</b> | <b>-6.06</b>                  | no max, no root-find             |

Comparisons of this kind are associated with Aruoba, Fernández-Villaverde & Rubio-Ramírez (2006), which sets VFI, perturbation, Chebyshev projection, finite elements and PEA against each other. **The table above is our own run**, and it covers EGM and time iteration, which that paper does not.

So this one is ours, on one machine, at one calibration, with the script in the repository. **Cite a benchmark table to the run that produced it.**

# Takeaways

- The bill factorises into **sweeps** (which belong to  $\beta$ ) and **cost per sweep** (which belongs to the maximization). They are attacked by different tools.
- **Howard** cut 1,352 maximizations to 29 at identical accuracy — a  $22\times$  speed-up that does not move the answer at all.
- **Monotonicity halved the comparisons and quadrupled the wall clock**, because a ratchet is sequential and a full search vectorises. Report both numbers; in a compiled language the sign flips.
- Continuous choice is the first change that buys *accuracy* rather than speed:  $10^{-1.55} \rightarrow 10^{-2.77}$ .
- **EGM wins on both**: 38 times faster and four and a half decades more accurate, with zero root-finding — but only if the grid is on **cash on hand**. Keep it in  $k$  and the root-find comes back.
- Report residuals **off** the solution grid, and cite a benchmark table to the run that produced it.

# Bridge: What You Owe, and What Comes Next

**Session 4 in one sentence.** Every crude choice of Session 3 had a repair; we built each one and measured what it bought — and the largest single gain came from *reformulating the state*, not from a better algorithm.

- **4.B5** puts the whole toolkit into one program: the stochastic model solved again, line by line, against the crude solver.
- **Lab L04** runs all of it: the root-finder bake-off, golden section, the interpolation shape tests, quadrature, Tauchen against Rouwenhorst, and the accelerated RBC.
- **Homework 1, Part B** is out today, due October 22 — and Part A is due the same day.

## Next week (S5)

Numerical Methods II. Everything so far has put values on a grid. **Perturbation** throws the grid away and expands around the steady state; **projection** replaces  $N$  separate values with a handful of basis coefficients.

Both are then run on *this* model, side by side. Aiyagari follows in Session 6.

And two threads from 4.A3 are left open on purpose: **CG and SGD** return in **S7**, as the optimizers that actually train a network, and the **Metropolis–Hastings** rule returns in **S9**, where the object being sampled is a posterior rather than a maximum.